

수륙양용 입무통제 SW 메뉴얼



목차



운용통제 #1



운용통제 #2



운용통제 #3

0 제어프로그램

- 운용통제 제어 기능
- 임무통제 제어 기능

1 소프트웨어 통합 ICD

2 SW 제작 현황

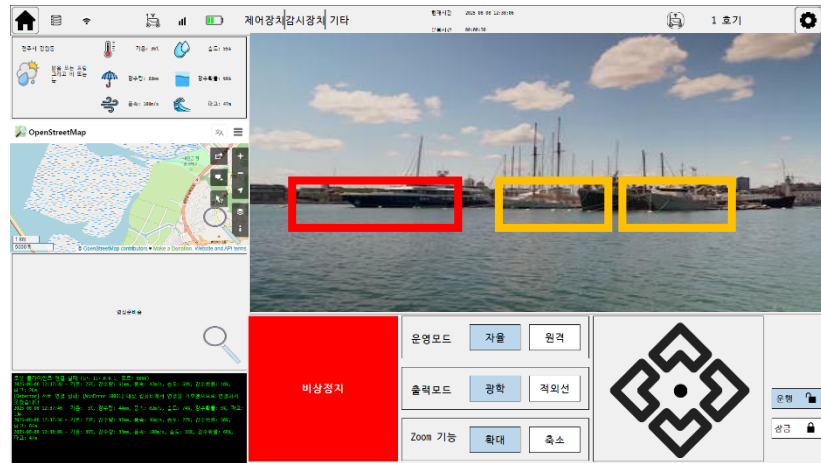
3 제작 세부 현황

4 시뮬레이터 원격 제어

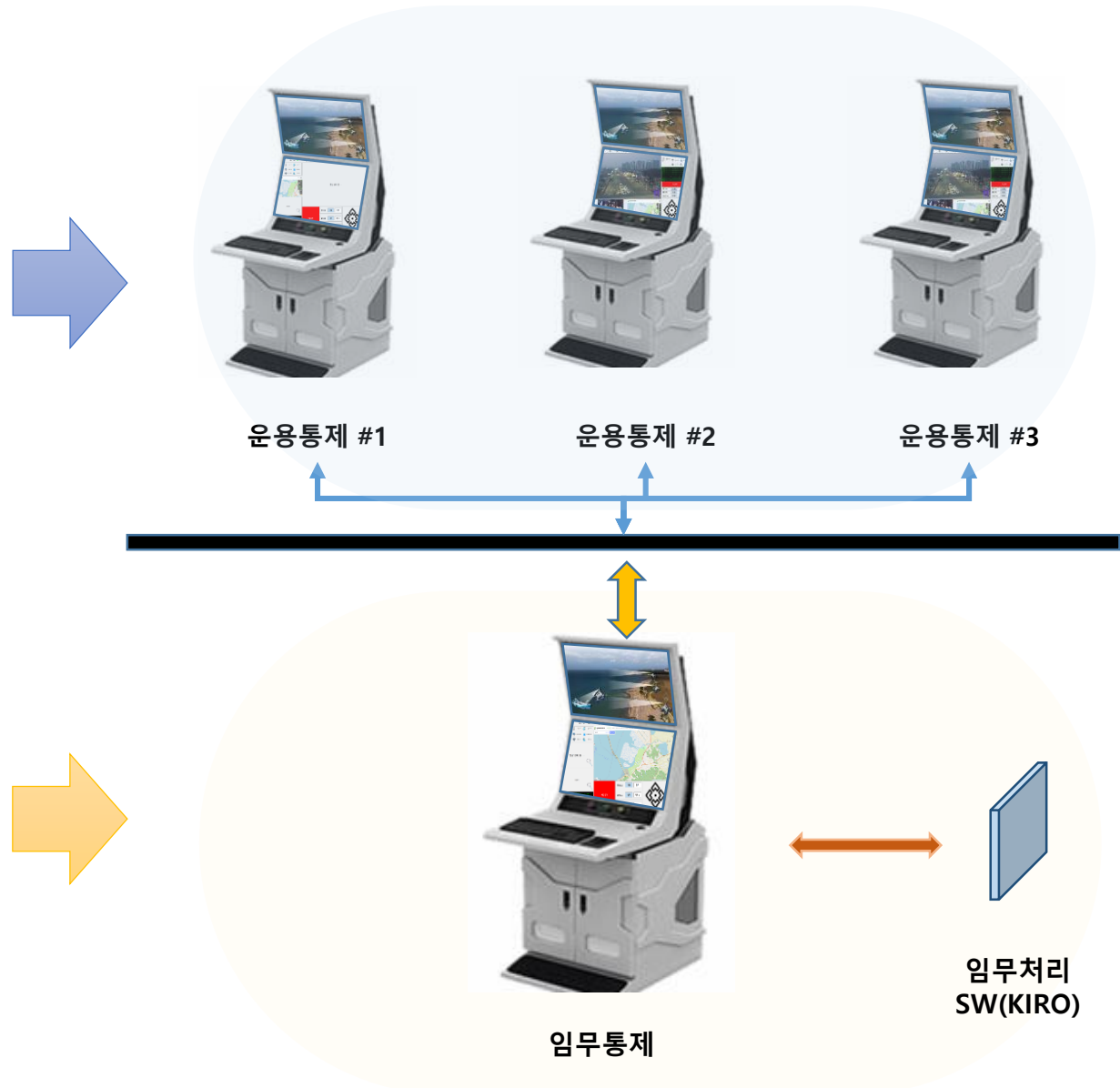
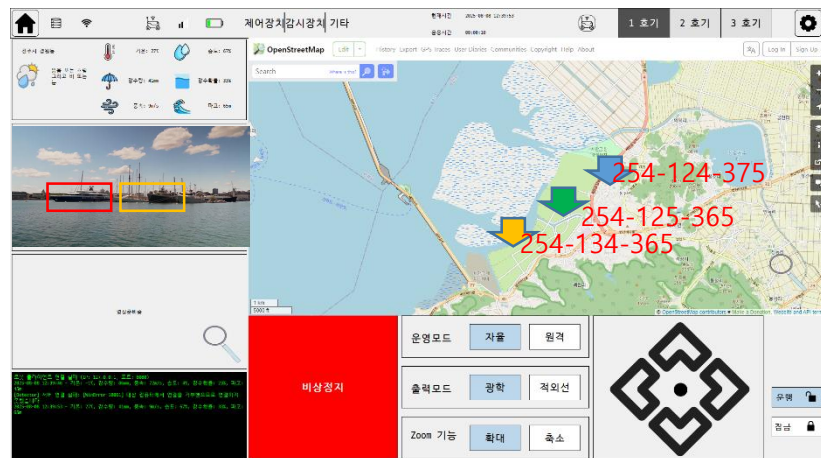
04 SW 설정파일 관리

0. 운용통제 / 임무통제 SW 구분

운용통제 제어 SW

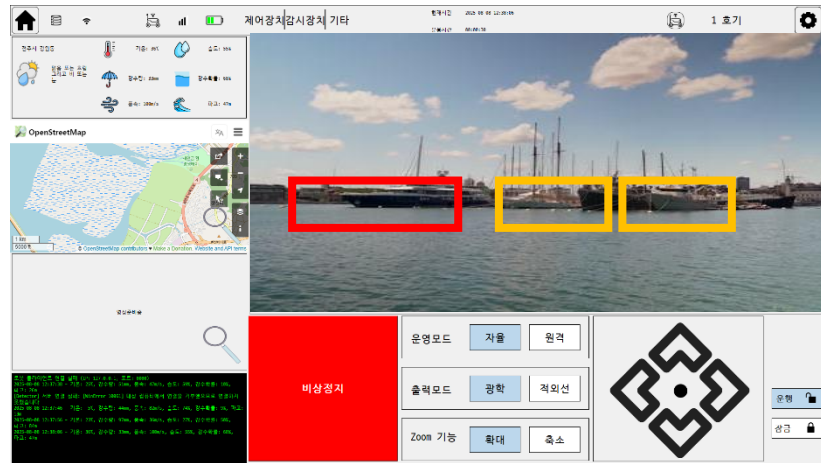


임무통제 제어 SW



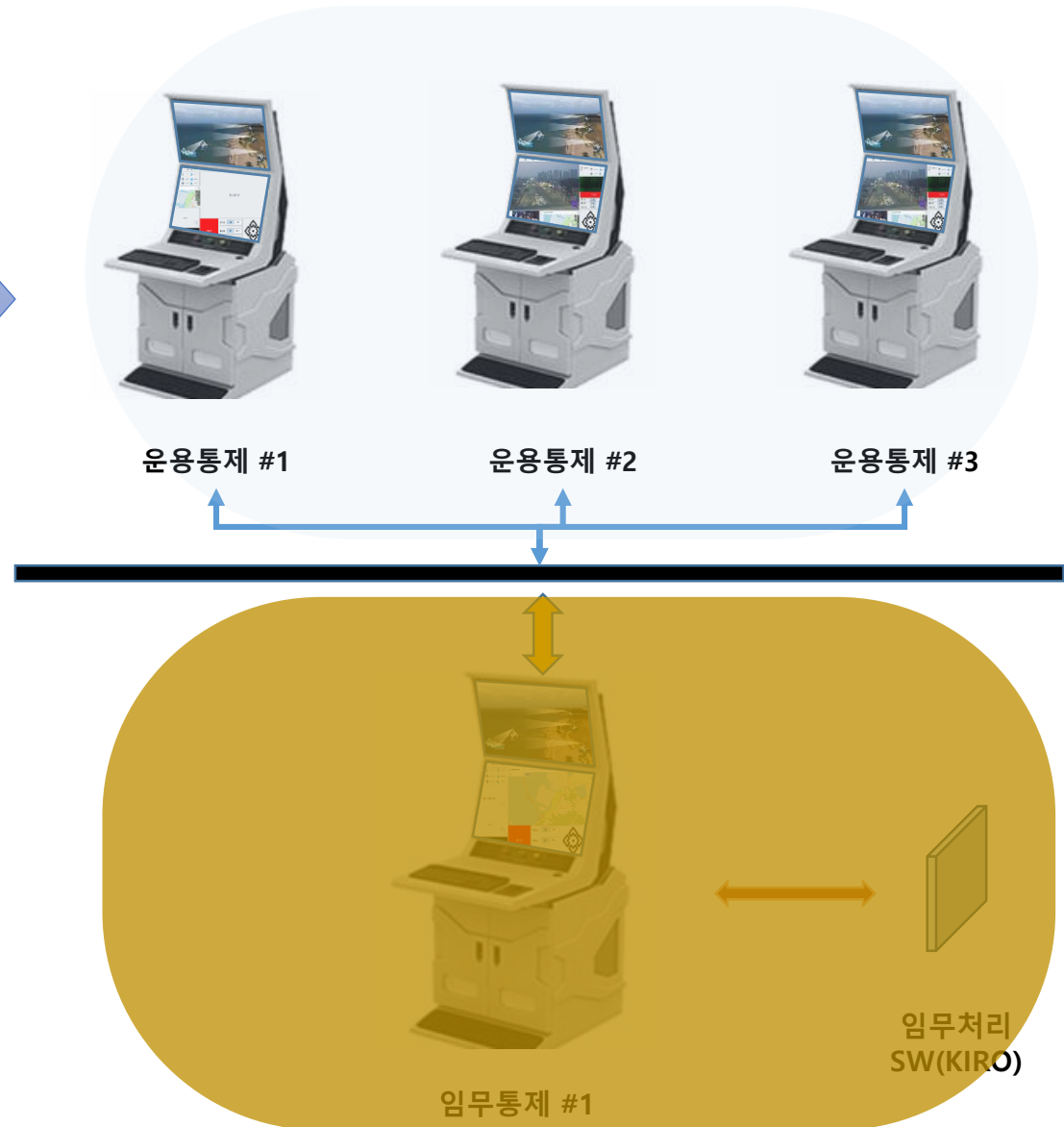
0. 운용통제 / 임무통제 SW 구분

운용통제 제어 SW



<기능>

- <로봇 #1 제어> : 운영제어 자율/수동 이동, 회기 등
- <로봇 영상 모니터링> : 로봇영상 제어 및 모니터링
- <분석영상 모니터링> : 영상 분석, 정보 제공
- <위치표시> : 위치정보 표시



0. 운용통제 / 임무통제 SW 구분

운용통제 제어 SW

<기능>

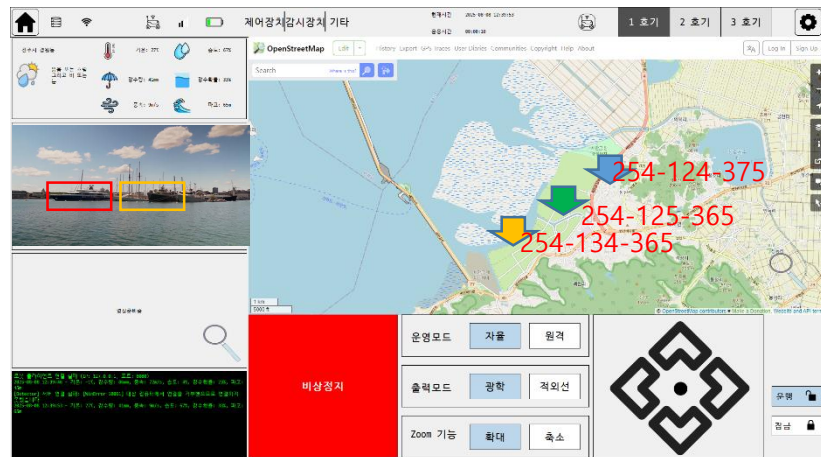
<로봇> 1~3 선택 및 영상 모니터링

<로봇> 선택 로봇 제어 및 모니터링

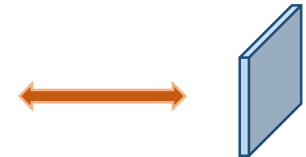
<로봇 영상 모니터링> : 1~3호기 위치 표시 및 방향 표시

<분석영상 모니터링> : 운용 통제 기능 제공

임무통제 제어 SW



임무통제 #1



임무처리
SW(KIRO)

0. 운용통제 제어프로그램 구분

타이틀 메뉴 기능

기능 : 시작, 설정, 종료

시작 : 제어 프로그램으로 이동

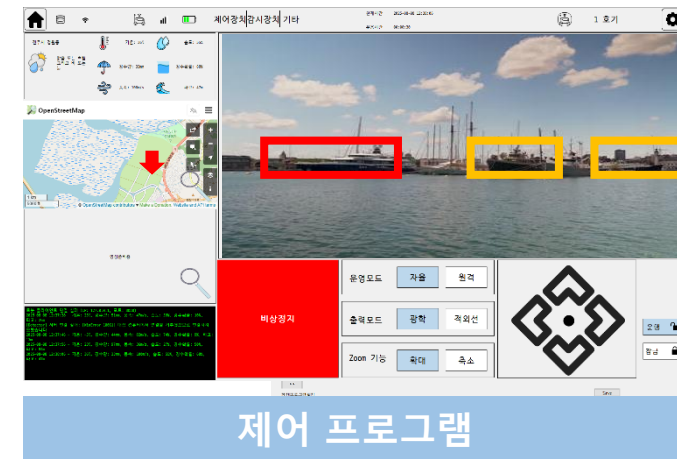
- 로봇 영상, 분석영상, 위치/방향 모니터링, 로봇 제어 기능 제공, 로봇 현황 및 하드웨어 정보 표시

설정 : 로봇 영상 주소 등 옵션 선택 저장

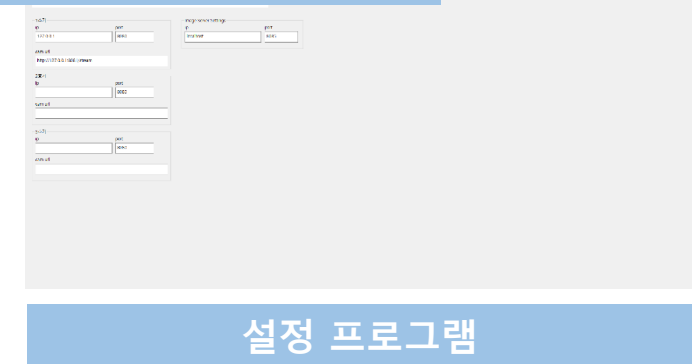
종료 : 프로그램 종료 확인



시작



설정



0. 운용통제 제어프로그램 기능

운용통제 제어기능 - 1번 상태바 기능

기능 : 로봇 영상 모니터링 및 제어 기능 제공

기능 : 로봇 영상, 분석영상, 위치/방향 모니터링, 로봇 제어 기능 제공, 로봇 현황 및 하드웨어 정보 표시



제어 프로그램

번호	기능	설명
1	로봇 기능 상태 표시	
	네트워크 감도	광범위 네트워크 상태 표시
	와이파이 감도	와이파이 감도 표시
	배터리 정보	로봇 배터리 정보
	로봇 상태 표시	로봇 운영 시스템 상태
	제어장치 정보	제어 시스템 상태
	감시장치정보	카메라사 상태
	날짜표시	날짜표시
	운영시간표시	운용시작시간, 현재시간, 누적시간
	선택 로봇 표시	현재 로봇 번호
	로봇 운용 표시	로봇이 이동중, On상태 정지중

0. 운용통제 제어프로그램 기능

운용통제 제어기능 - 분석영상 모니터링 뷰, 날씨 정보, 로봇 위치 방향 정보 뷰

기능 : 로봇 영상 모니터링 및 제어 기능 제공

기능 : 로봇 영상, 분석영상, 위치/방향 모니터링, 로봇 제어 기능 제공, 로봇 현황 및 하드웨어 정보 표시



번호	기능	설명
2	로봇 모니터링 영상 분석 오브젝트 객체 인식	
	분석영상	AI 객체 분석
	분석 데이터 표시	객체 구분 표시
3	날씨 정보 표시	
	날씨표시	비, 바람, 파도, 습도, 온도
4	로봇 위치 방향 표시	
	구분 표시	선택 로봇 표시
	보는 방향 표시	카메라 보는 방향 표시
	이동 표시	이동하고 있는 방향표시
	로봇 위치 표시	좌표 표시
	다른 로봇 위치	다른 로봇 위치 표시

제어 프로그램

0. 운용통제 제어프로그램 기능

운용통제 제어기능 – 로봇 카메라 영상 뷰, 네트워크 정보 표시, 로봇 수동 원격 제어 기능
기능 : 로봇 영상 모니터링 및 제어 기능 제공

기능 : 로봇 영상, 분석영상, 위치/방향 모니터링, 로봇 제어 기능 제공, 로봇 현황 및 하드웨어 정보 표시



번호	기능	설명
5	로봇 모니터링 영상 화면	
	카메라 영상	카메라 원본 영상 모니터링 뷰
6	로보소 네트워크 정보 머니터링 정보창	
	네트워크 정보	로봇, 임무통제, 등 연결상태 표시
7	로봇 제어	
	운용모드	자율주행, 원격 주행 제어
	광학모드	광학, 적외선 카메라 기능 제어
	줌기능	카메라 줌기능
	비상정지	운용중 긴급 사항 운영 중지
	제어기능 잠금	제어기능 잠표 기능
	로봇 방향제어	로봇 이동을 원격으로 제어

제어 프로그램

0. 임무통제 제어프로그램 기능

운용통제 제어기능 - 1번 상태바 기능

기능 : 로봇 영상 모니터링 및 제어 기능 제공

기능 : 로봇 영상, 분석영상, 위치/방향 모니터링, 로봇 제어 기능 제공, 로봇 현황 및 하드웨어 정보 표시



설

번호	기능	설명
1	로봇 기능 상태 표시	
	네트워크 감도	광범위 네트워크 상태 표시
	와이파이 감도	와이파이 감도 표시
	배터리 정보	로봇 배터리 정보
	로봇 상태 표시	로봇 운영 시스템 상태
	제어장치 정보	제어 시스템 상태
	감시장치정보	카메라사 상태
	날짜표시	날짜표시
	운영시간표시	운용시작시간, 현재시간, 누적시간
	선택 로봇 표시	현재 로봇 번호
	로봇 운용 표시	로봇이 이동중, On상태 정지중
선택	호기 선택	현재 운영중인 로봇 호기 선택

0. 임무통제 제어프로그램 기능

운용통제 제어기능 - 분석영상 모니터링 뷰, 날씨 정보, 로봇 위치 방향 정보 뷰

기능 : 로봇 영상 모니터링 및 제어 기능 제공

기능 : 로봇 영상, 분석영상, 위치/방향 모니터링, 로봇 제어 기능 제공, 로봇 현황 및 하드웨어 정보 표시



번호	기능	설명
2	로봇 모니터링 영상 분석 오브젝트 객체 인식	
	분석영상	AI 객체 분석
	분석 데이터 표시	객체 구분 표시
3	날씨 정보 표시	
	날씨표시	비, 바람, 파도, 습도, 온도
4	로봇 위치 방향 표시	
	구분 표시	선택 로봇 표시
	보는 방향 표시	카메라 보는 방향 표시
	이동 표시	이동하고 있는 방향표시
	로봇 위치 표시	좌표 표시
	다른 로봇 위치	다른 로봇 위치 표시

설

0. 임무통제 제어프로그램 기능

운용통제 제어기능 – 로봇 카메라 영상 뷰, 네트워크 정보 표시, 로봇 수동 원격 제어 기능
기능 : 로봇 영상 모니터링 및 제어 기능 제공

기능 : 로봇 영상, 분석영상, 위치/방향 모니터링, 로봇 제어 기능 제공, 로봇 현황 및 하드웨어 정보 표시



설

번호	기능	설명
5	로봇 모니터링 영상 화면	
	카메라 영상	카메라 원본 영상 모니터링 뷰
6	로보소 네트워크 정보 머니터링 정보창	
	네트워크 정보	로봇, 임무통제, 등 연결상태 표시
7	로봇 제어	
	운용모드	자율주행, 원격 주행 제어
	광학모드	광학, 적외선 카메라 기능 제어
	줌기능	카메라 줌기능
	비상정지	운용중 긴급 사항 운영 중지
	제어기능 잠금	제어기능 잠표 기능
	로봇 방향제어	로봇 이동을 원격으로 제어

0. 운용통제 설정 (연결 주소 셋팅)

운용통제 제어기능 - 로봇 카메라 연결 설정, AI 분석 시스템 연결, 로봇 정보 연결

호기별 접속 ip url

로봇제어 주소

cam 접속 주소

<<

제어프로그램설정

사운드설정

화면설정

1호기

ip 127.0.0.1 port 8080

cam url http://127.0.0.1:8081/stream

2호기

ip port 8080

cam url

3호기

ip port 8080

cam url

Save

설정후 집모양(홈버튼으로 나갔다 들어오면 바로 적용 됩니다.)

0. 운용통제 설정 (연결 주소 셋팅)

운용통제 제어기능 - 로봇 카메라 연결 설정, AI 분석 시스템 연결, 로봇 정보 연결



<< Save

제어프로그램설정

사운드설정	켜	끔
화면설정	켜	끔

1호기

ip	port
127.0.0.1	5000
cam url	
rtsp://ailab.miso.center:21054/test1	

2호기

ip	port
	5001
cam url	
rtsp://ailab.miso.center:21054/test2	

3호기

ip	port
	5002
cam url	

Image Server Settings

ip	port
106.255.251.154	21035

GIST -> 보낸주소

Gist 로 보내면 영상을 분석해서 다시 받아 그려주는 형태 입니다.

1. 소프트웨어 통합 ICD

1.1 통신 소켓 연결

1.1.1 로봇플랫폼 통신

- 소켓 Server 역할 예상(지휘통제소 SW 접속 방식)

1.1.2 감시장비 통신

- 영상처리 RTSP – [G Streamer] 활용 (멀티캐스트 송신)
 - : RTSP 라이브러리 확인 및 테스트 필요
 - 현재 RTSP 관련 변경 및 추가 협의 진행 없음
- 소켓 Server 고정(변경 불가)
 - : 무선 테스트 진행 X → 지연시간 테스트 필요
 - : 유선 테스트(내부 300ms) → GIST 연동 임시 테스트 약 500ms 이상

1.1.3 영상처리 통신

- 영상처리관련 협의 진행 사항
 - : 표준영상 데이터를 적용한 영상출력 진행
 - : 정의된 영상프로토콜 데이터 영상 변경 시뮬레이션 적용 완료

1.1.4 군집운용 통신

- 추후 임무통제장치 SW 적용 소켓 통신 내부 임무시나리오 명령 수행
 - : 시뮬레이터 개발을 통한 내부 테스트 진행후 군집운영 통신 적용 제어 진행

1.2 ICD 초안 관련

- 현재 ICD 초안 내부 포맷 및 주소 값 지정 관련 검토 필요
 - : 변경 및 진행 현황 대비 추가 필요 사항

2. 소프트웨어 제작 현황

2.1 임무통제장치 SW

- 현재 GUI 설계(안) 기준 제작 현황 확인 필요
 - : 임무통제 제어 프로토콜을 적용한 시뮬레이터 개발 제어 진행
 - : 가상의 시뮬레이터를 활용한 테스트 진행
 - : 정의된 프로토콜을 이용한 제어신호 적용 테스트 진행 완료
- 시뮬레이터 추가 개발
 - : 임무통제 서버 개발, 원격제어 클라이언트개발, 기관정의된 프로토콜적용
- 패킷 통신 시뮬레이션을 통한 원격 관제시스템 개발
- 설정 파일 관리 시스템 개발
- 추후 타 기관 SW 통합 진행 필요
 - : 내부 소켓 활용 [로봇군집운용 - 임무시나리오명령] 관련 수행 예정

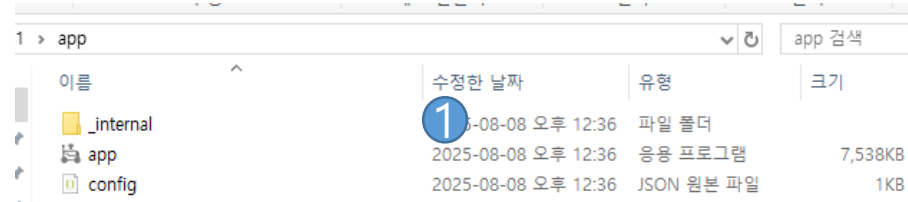
2.2 휴대용조종장치 SW

- 현재 GUI 설계(안) 기준 제작 현황 확인 필요
 - : 운영통제 제어 와 동일한 기능 및 제어 SW 설계 진행
 - : 모바일 프로토콜 적용한 제어 프로그램 개발 진행 예정

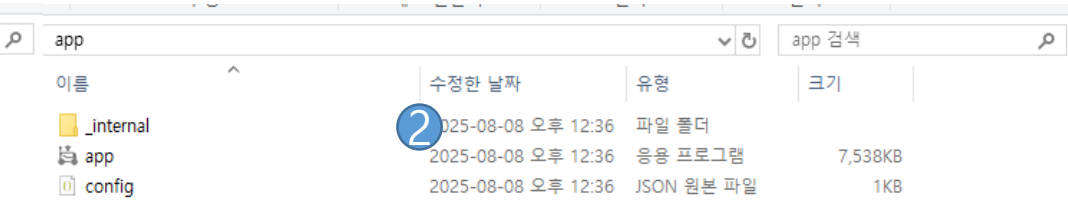
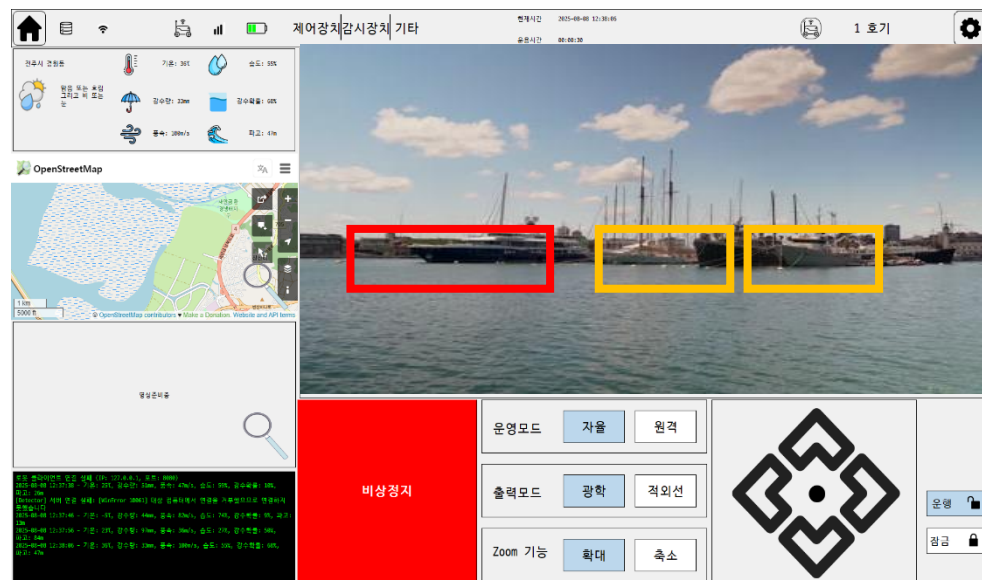
3. 소프트웨어 제작 세부 현황

3.0 임무통제장치 SW 시뮬레이터 테스트 환경

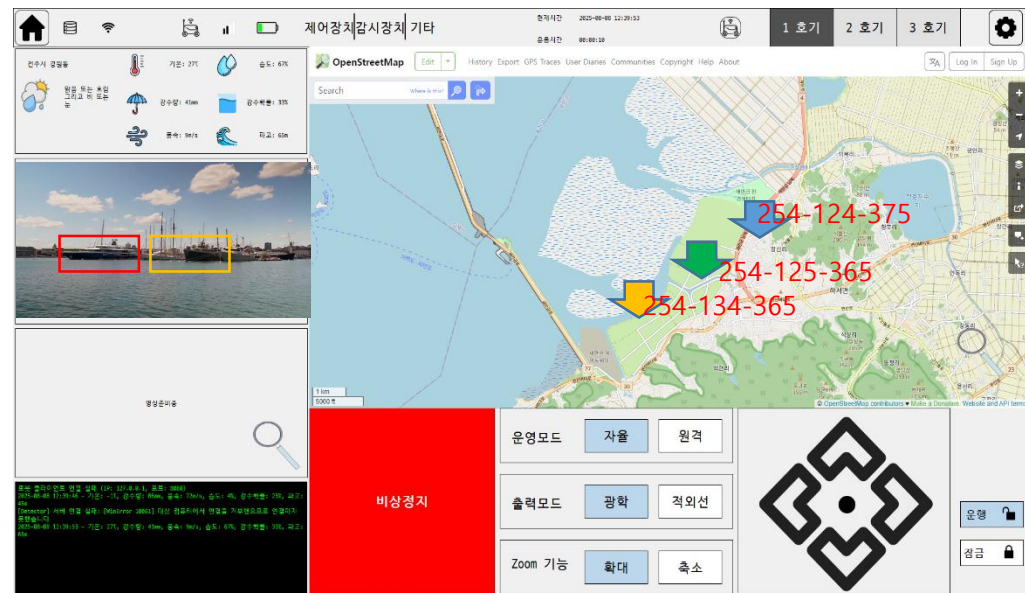
-앞축파일 2개 설치 후 1번 실행 nagibot->simulator->car_simulator, 실행 2번 nagibot->control->app실행



운용통제 제어 SW



임무통제 제어 SW



3. 소프트웨어 세부 현황

3.1 임무통제장치 SW 제어 시뮬레이터 시스템 아키텍처

시뮬레이터 (서버 측)

- **물리 시뮬레이션**: MuJoCo 엔진을 사용하여 차량의 동작을 계산합니다. XML 모델 파일(예: citeturn0file1, citeturn0file2)을 통해 차량의 형상, 관절, 액추에이터, 센서 등이 정의.
- **통신 서버**: TCP 소켓을 열어 클라이언트의 연결을 대기하며, 수신된 주행 제어 명령을 해석하고 차량 동작 반영.
- **렌더링**: Pygame 기반의 렌더러를 이용하여 시뮬레이션 장면을 실시간으로 화면 출력.

원격 제어 클라이언트 (클라이언트 측)

- **GUI 인터페이스**: Pygame을 이용하여 사용자에게 차량 상태(속도, 위치, 바퀴 상태 등)를 표시하고, 키보드 입력으로 주행 명령(전진, 후진, 좌우 회전) 전송. citeturn0file0
- **네트워크 통신**: TCP 소켓을 통해 시뮬레이터와 양방향 통신을 수행하며, JSON 형식으로 명령과 상태 데이터 Sin, Ack.

통신 프로토콜

- 정찰로봇 프로토콜은 Little-Endian 방식으로 패킷을 직렬화하며, 패킷의 헤더, 데이터, 접미사를 포함하는 구조 설계. (자세한 내용은 citeturn0file3 참고)

3. 소프트웨어 세부 현황

3.2 임무통제장치 SW 제어 프로토콜 설계

패킷 구조

- 패킷은 시작 프리픽스(PREFIX), 송신자/수신자 ID, 시퀀스 번호, 전송 종류(SendType), 내용 종류(ContentType), 데이터 길이, 데이터, 접미사(SUFFIX) 구성.
- 데이터 부분에는 센서 상태 정보나 주행 제어 명령과 같이 구조체를 이진 형식으로 직렬화 값 포함.

구조체 정의

- **SensorStatus**: 센서 ID, 상태, 온도, 배터리 정보를 포함하며, 이를 이진 형식으로 변환하여 패킷 데이터 포함.
- **DriveControl**: 차량의 속도, 회전각(라디안), 운용 모드(자율/수동 등)를 포함하는 제어 명령으로, 주행 제어 명령 패킷의 데이터 전송.

패킷 생성 및 파싱 함수

- 센서 상태 및 주행 제어 명령에 대해 각각 패킷 생성(create_sensor_status_packet, create_drive_control_packet) 및 파싱(parse_sensor_status_data, parse_drive_control_data) 기능을 제공하여, 통신의 신뢰성 확보.

3. 소프트웨어 세부 현황

3.3 임무통제장치 SW 시뮬레이터 개발

모델 로드 및 시뮬레이션

- MuJoCo의 XML 모델 파일(예: citeturn0file1)을 이용하여 차량 모델을 로드합니다. 차량은 freejoint를 포함한 본체와 좌우 바퀴로 구성되어 있으며, tendons를 통한 주행 및 회전 제어 구현.
- 시뮬레이션 루프에서는 MuJoCo의 `mj_step` 함수를 반복 호출하여 물리 계산을 수행하며, 이를 통해 차량의 위치, 방향, 속도 등의 상태 업데이트.

통신 및 제어 명령 처리

- TCP 서버를 열어 클라이언트의 연결을 대기하고, 수신된 주행 제어 명령을 파싱하여 차량 액추에이터 제어 신호 변환.
- 상태 전송 함수(`send_vehicle_state`)는 차량의 센서 데이터를 패킷으로 직렬화하여 클라이언트 전송.
- 인덱스 접근 방식은 모델의 내부 구조에 맞게 `jnt_qposadr`와 `jnt_dofadr`를 사용하여 올바른 데이터를 참조 구현.

렌더링

- Pygame을 사용하여 실시간 렌더링을 수행하며, 시뮬레이터의 창 종료 이벤트를 통해 시뮬레이션 및 통신 루프를 정상적으로 종료 설계.

3. 소프트웨어 세부 현황

3.4 임무통제장치 SW 원격 제어 클라이언트 개발

GUI 및 사용자 입력

- Pygame 기반의 GUI를 통해 차량 상태(위치, 속도, 바퀴 위치 등)를 시각적으로 표시하며, 키보드 입력을 통해 주행 명령(전진, 후진, 좌/우 회전) 제어.
- 실시간 입력에 따라 명령 값이 변화하고, 일정 시간 간격마다 명령이 TCP 소켓을 통해 시뮬레이터 전송.

상태 수신 및 업데이트

- 별도의 수신 스레드를 통해 시뮬레이터로부터 상태 메시지를 받아 처리하고, GUI에 해당 정보 업데이트.
- 상태 데이터는 JSON 형식으로 전송되며, 이를 파싱하여 차량의 현재 상태 반영.

유연한 운영 모드

- GUI 모드와 콘솔 모드(텍스트 기반 입력) 모두 지원하여, 다양한 환경에서 시스템을 테스트하고 운영 설계.

3.5 임무통제장치 SW 기술 스택

- 언어 및 라이브러리: Python, NumPy, struct, threading, socket
- 시뮬레이션 엔진: MuJoCo
- 그래픽 및 GUI: Pygame, PySide6 (일부 UI 모듈)
- 프로토콜 구현: 사용자 정의 정찰로봇 프로토콜 (이진 패킷 직렬화)
- 데이터 직렬화: JSON (클라이언트 상태 전송)

4. 소프트웨어 시뮬레이션 원격 제어

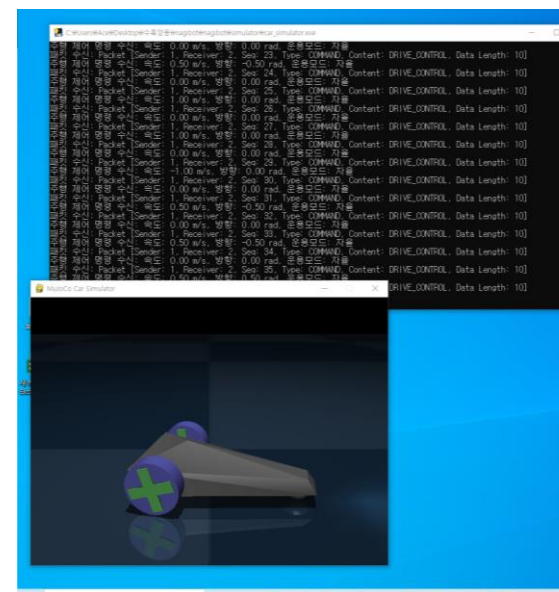
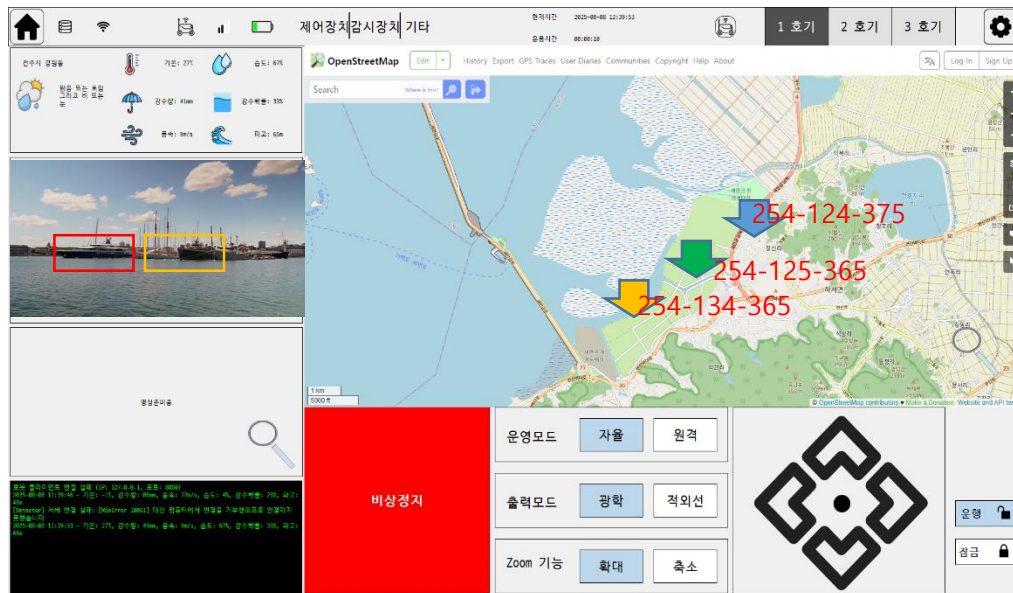
4.1 임무통제장치 SW 시스템 구성 및 아키텍처

모듈 분리

- 패킷 통신 모듈 (protocol.py): 통신 프로토콜의 직렬화 및 파싱을 담당.
- 원격 관제 GUI 모듈 (mainForm.py): PySide6를 활용해 사용자 인터페이스, 영상 스트림 처리, 상태 업데이트, 및 제어 명령 전송 기능을 구현.

통신 흐름

- 정찰로봇(클라이언트)과 지휘통제소(서버) 간에 패킷 단위로 데이터를 주고받으며, 시뮬레이션 환경에서는 가상 센서 데이터와 영상 스트림을 사용하여 전체 시스템 동작을 검증.



4. 소프트웨어 시뮬레이션 원격 제어

4.2 임무통제장치 SW 패킷 통신 프로토콜 구현

패킷 구조

- 헤더 구성

- 프리픽스(PREFIX: 0xAABB)와 접미사(SUFFIX: 0xEDFF) 패킷 명시.
- 송신자/수신자 ID, 순번(Sequence Number), 전송 타입(Send Type), 내용 타입(Content Type), 데이터 길이(Data Length) 포함.

- 데이터 부분:

- 센서 상태 정보, 주행 제어 명령 등 실제 제어 및 상태 데이터 담음.

직렬화/파싱

- Python의 `struct` 모듈을 사용하여 Little-Endian 방식으로 데이터를 직렬화(`to_bytes()`) 및 파싱(`from_bytes()`) 처리.
- 센서 정보(SensorStatus)와 주행 명령(DriveControl)을 위한 별도의 클래스와 메서드를 구현하여 각 데이터 구조체 단위 변환.

오류 검증

- 프리픽스, 접미사, 데이터 길이 등의 검증 로직을 포함하여 패킷 무결성 확보.

4. 소프트웨어 시뮬레이션 원격 제어

4.3 임무통제장치 SW GUI 및 영상 처리 구현

PySide6 기반 사용자 인터페이스

- 다중 화면 처리

- 메인 카메라 화면과 서브 카메라 화면을 별도 스레드(VideoThread) 통해 RTSP 스트림 처리.
- OpenStreetMap URL을 사용하여 지도 정보를 웹 뷰(QWebEngineView) 표시.

- 실시간 상태 업데이트

- StatusUpdateThread를 통해 주기적으로 센서 데이터, 네트워크, 배터리 상태, 및 시간 정보 업데이트 기록.

- UI 구성 요소

- 버튼, 라벨, 텍스트 박스 등 다양한 위젯을 활용해 제어 명령, 상태 표시, 폰트 적용 및 테마 변경 기능 구현.

스레드 기반 처리

- 영상 데이터 처리와 UI 업데이트를 별도의 스레드에서 실행하여 메인 스레드의 블로킹 없이 실시간 처리 보장.

4. 소프트웨어 시뮬레이션 원격 제어

4.4 임무통제장치 SW 로봇 클라이언트 통신 및 제어 명령 처리

로봇 클라이언트 모듈

- `robot_client` 모듈을 통해 로봇과의 연결을 관리하며, 연결 성공 시 센서 데이터 업데이트와 제어 명령 전송 기능 활성화.

키 및 버튼 이벤트 처리

- 각 방향키(전진, 후진, 좌회전, 우회전) 및 비상 정지 버튼에 대해 눌림/해제 이벤트 처리.
- 이벤트 발생 시 대응하는 드라이브 제어 명령을 생성하고, 해당 명령 패킷 직렬화 전송.

드라이브 제어 패킷 생성

- DriveControl 구조체를 통해 속도, 방향, 운용 모드를 지정하고, 이를 바탕으로 주행 제어 패킷 생성 로봇 전송.

4.5 임무통제장치 SW 시뮬레이션 환경 및 테스트

가상 데이터 생성

- 센서 데이터(온도, 강수량, 풍속 등)는 난수로 생성 시스템 응답성 검증.
- 실제 RTSP 스트림 대신 테스트용 비디오 스트림 영상 처리 기능 확인.

패킷 통신 검증

- 패킷의 직렬화 및 파싱 과정을 반복 테스트하여 데이터 무결성 오류 검출 기능 검증.
- 센서 상태와 주행 명령의 각 전송 및 수신 과정 로그 실시간 모니터링.

UI 반응성 테스트

- 다중 스레드 환경에서 영상 스트림, 상태 업데이트, 지도 로딩 등 각 기능 동기적 정상 작동 확인.

5. 소프트웨어 설정파일 관리 시스템 구현

5.1 임무통제장치 SW ConfigManager 모듈 구현

기본 구성 설정 데이터 구조:

- 최대 3대의 차량 정보를 관리하며, 각 차량은 IP, 포트, 카메라 URL 저장.
- 추가로 전역 설정인 사운드 활성화(isSoundOn)포함.

```
self.config = {  
    "cars": [  
        {  
            "ip": "127.0.0.1",  
            "port": 8080,  
            "camUrl": "http://127.0.0.1:8081/stream"  
        },  
        { "ip": "", "port": 8080, "camUrl": "" },  
        { "ip": "", "port": 8080, "camUrl": "" }  
    ],  
    "isSoundOn": True  
}
```

5. 소프트웨어 설정파일 관리 시스템 구현

5.2 임무통제장치 SW ConfigManager 모듈 구현

JSON 기반 파일 입출력

- 저장(save_config) `json.dump`를 사용하여 사람이 읽기 좋은 인덴트 형식으로 설정값을 JSON 파일 저장.
- 불러오기(load_config) 설정 파일이 존재할 경우, JSON 파일을 읽어 기본 설정 딕셔너리와 병합.
- 파일이 없거나 예외가 발생하면 기본 설정 적용 설계.

개별 설정 접근 및 변경

- Getter/Setter 메서드

- 차량의 IP, 포트, 카메라 URL을 개별적으로 조회(get_car_ip, get_car_port, get_car_cam_url) 및 설정(set_car_ip, set_car_port, set_car_cam_url).
- 차량 전체 정보를 다루기 위한 get_car_info와 set_car_info 메서드 제공.

- 유효성 검증

- 차량 인덱스가 0~2 범위 내에 있는지 검사하여 잘못된 접근 시 ValueError 발생.
- 데이터의 무결성을 유지, 잘못된 설정값 입력 방지.

5. 소프트웨어 설정파일 관리 시스템 구현

5.3 임무통제장치 SW 설정 폼 GUI 구현(setupForm 모듈)

UI 구성 및 초기 데이터 반영

- PySide6 기반 폼 setupForm.py는 PySide6를 활용한 UI 위젯으로, UI.setupForm 모듈을 상속받아 폼 레이아웃 구성.
- 초기 설정 로드 폼 생성 시 ConfigManager 인스턴스를 생성하고, load_config() 호출, JSON 파일 설정값 불러옴.
- 설정값을 각 텍스트 에디터(예: textEditCar1ip, textEditCar1port, textEditCar1Camurl 등)에 반영되어 사용자가 현재 설정 확인.

사용자 입력 처리 및 저장

- 설정 변경 이벤트 처리
 - "저장" 버튼(pushButton_saveSetup) 클릭 시, onClick_btnSaveSetup 핸들 호출.
 - 이 핸들러는 각 텍스트 필드의 값을 읽어 ConfigManager의 setter 메서드를 통해 내부 설정 값 업데이트.
 - 변경된 설정은 save_config()를 호출하여 JSON 파일 다시 저장.
- 폼 전환 및 종료 뒤로가기" 버튼(btnBack)은 backSignal을 emit하여 폼 전환 기능을 지원, closeEvent에서도 closedSignal을 emit하여 폼 종료 시 후속 처리.

5. 소프트웨어 설정파일 관리 시스템 구현

5.4 임무통제장치 SW 통합 구현 흐름 및 특징

구현 흐름

- ConfigManager 생성 시 기본 설정이 정의되고, load_config()로 기존 설정 파일 불러옴.
- setupForm 폼에서 ConfigManager를 통해 각 차량 및 사운드 설정값을 UI 위젯 반영.
- 사용자가 텍스트 에디터를 통해 IP, 포트, 카메라 UR을 수정.
- 저장 버튼 클릭 시, 입력값이 setter 메서드를 통해 업데이트되고, save_config()가 호출되어 변경된 설정 JSON 파일 기록.

기술적 장점 및 확장성

- 모듈화 설정 관리 기능이 ConfigManager 클래스로 분리되어 있어, 다른 프로젝트나 모듈 재사용.
- 유지보수성 JSON 포맷을 사용하여 설정 파일이 사람이 읽기 쉽고, 수동 편집 가능 설계.
- 데이터 무결성 각 setter 메서드에서 인덱스 범위 체크 등 유효성 검증 로직을 포함하여, 잘못된 데이터 입력 방지.
- GUI와의 통합 PySide6 기반 UI에서 ConfigManager를 직접 호출하여 실시간으로 설정값을 반영하고, 변경 사항을 저장 편의성 확보.